

Divide and Conquer

Divide & Conquer

- 1. **Divide** the problem into smaller subproblems
- 2. **Conquer** the subproblems via recursive calls
- 3. **Combine** the subproblem answers...

An Intuitive Viewpoint

- Break the problem into smaller subproblems...
- Assume that some 'oracle' solves the smaller subproblems and gives you the solution
- Try to solve the original problem using these solutions to subproblem...

An Intuitive Viewpoint

- Break the problem into smaller subproblems...
- Assume that some 'oracle' solves the smaller subproblems and gives you the solution
- Try to solve the original problem using these solutions to subproblem...
- Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !

An Intuitive Viewpoint

- Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !
- But where is the oracle ? There is none !
- Just keep dividing the problem until you can solve using just simple primitive operations (also called Base Cases)...

An Intuitive Viewpoint

- Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !
- But where is the oracle ? There is none !
 - Just keep dividing the problem until you can solve using just simple primitive operations (also called Base Cases)...
 - The recursion paradigm keeps combining solutions to smaller subproblems starting with the base cases...

Divide and Conquer

- Classic example: merge sort
 - Input: 1, 3, 5, 2, 4, 6; Output: 1, 2, 3, 4, 5, 6
 - Naive approach could take $O(n^2)$ time.
 - Divide the problem into two (or more) subproblems of size $n/2...$

Divide and Conquer

- Classic example: merge sort
 - Divide the problem into two (or more) subproblems of size $n/2$
- Combine the solutions of the subproblems and build a solution for the original problem
- $T(n) = a T(n/2) + f(n)$
- Divide and conquer might give a running time improvement e.g., from $O(n^2)$ to $O(n \log n)$.

Divide and Conquer

- Counting Inversions in an Array:
 - Input: An array A of (distinct) natural numbers, suppose $1, 2, 3, \dots, n$ (for this lecture) in any arbitrary order.
 - Output: The number of inversions in the array = number of pairs (i, j) such that $i < j$, but $A[i] > A[j]$.
 - Example: $1, 3, 5, 2, 4, 6, \dots$
 - # of inversions = 3
 - $(3, 2), (5, 2), (5, 4)$

Divide and Conquer

- Application: E-commerce Websites : Collaborative filtering... Inversion counting is used to determine 'similarity' between two customers...
- Naive algorithm takes $O(n^2)$ time.
- Can we design an $O(n \log n)$ time algorithm, which outputs all inversions?

Divide and Conquer

- Suppose A is divided in to X and Y .
- Left inversion : Both (i,j) in X
- Right inversions : Both (i,j) in Y
- Split inversions : i in X , j in Y

Divide and Conquer

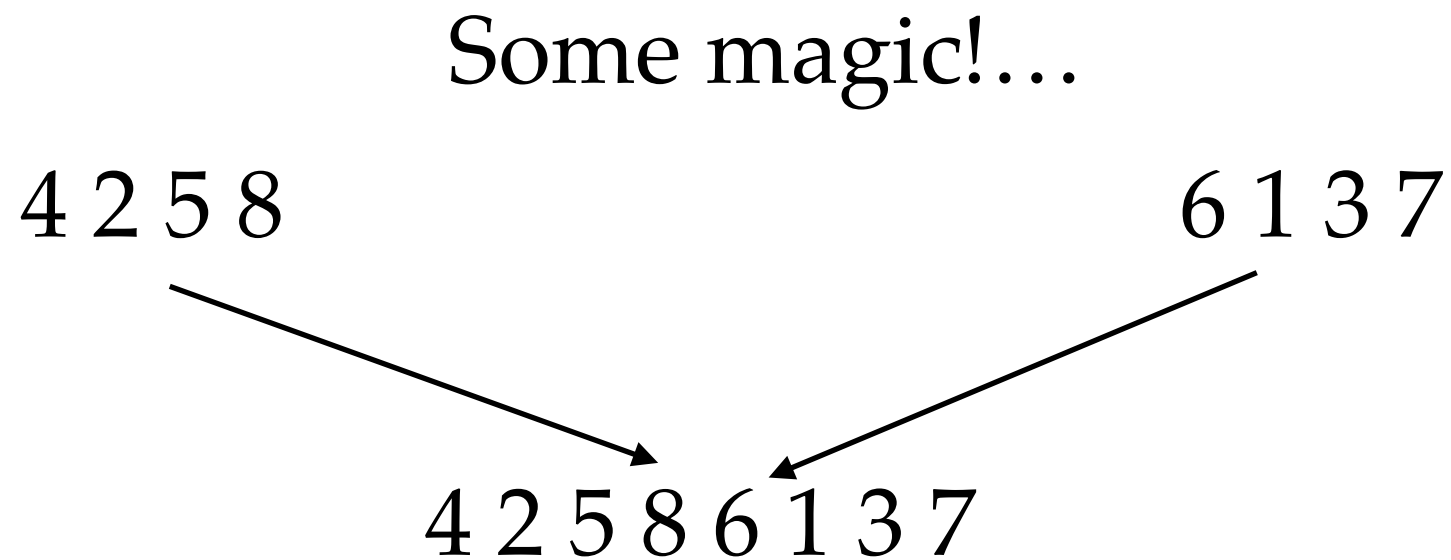
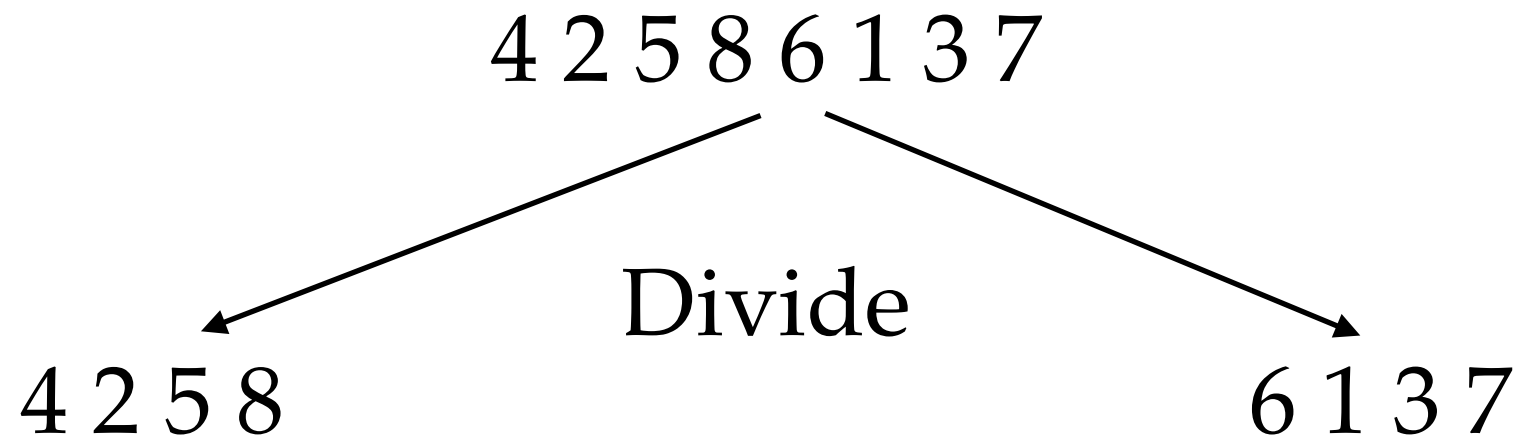
- Example: 1, 5, 3, 2, 4, 6
 - Left inversion : ($X(1), X(2)$)
 - Right inversions : None
 - Split inversions : $X(1), Y(0)$ $X(1), Y(1)$ and $X(2), Y(0)$

Divide and Conquer

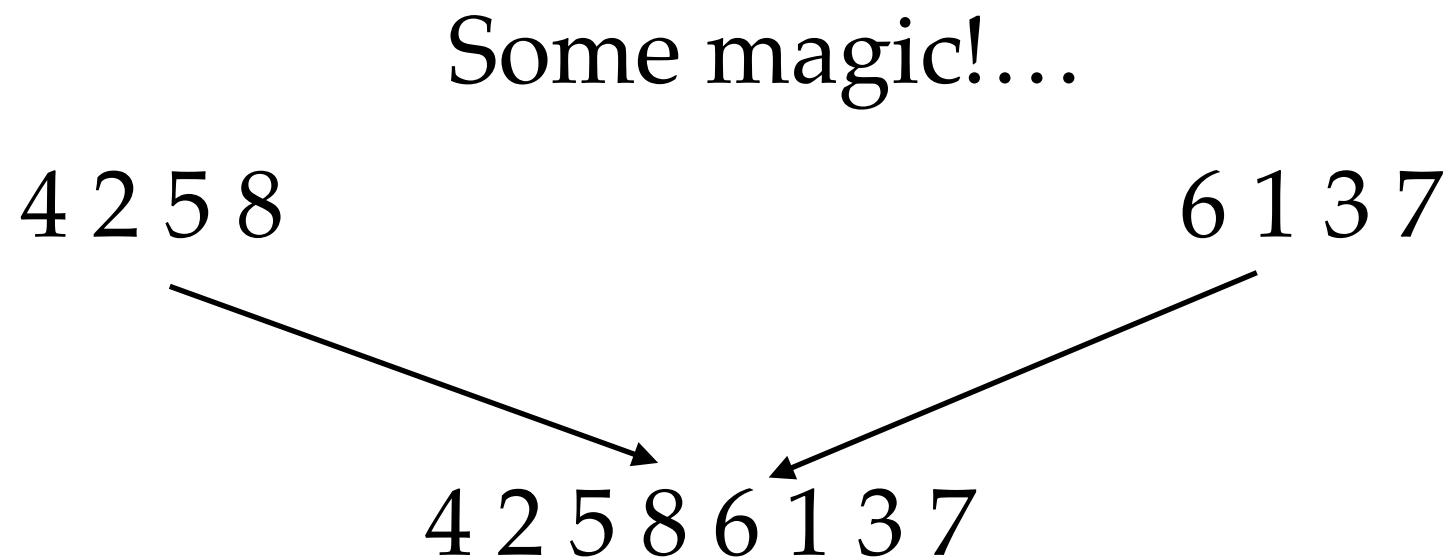
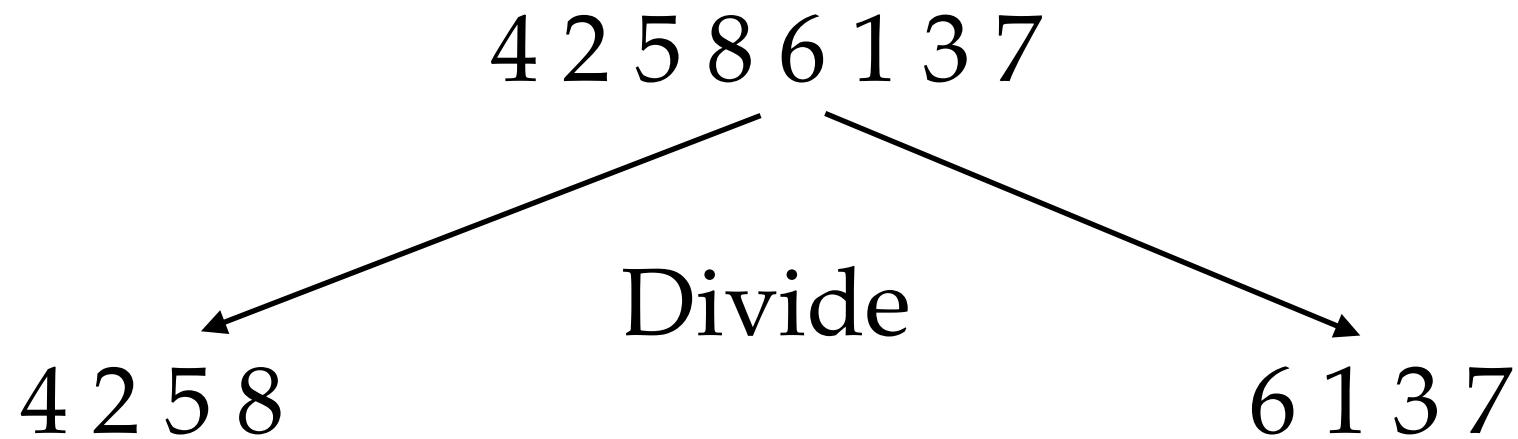
- Suppose A is divided into X and Y .
- An inversion pair (i, j) is :
 - Left inversion : Both (i, j) in X
 - Right inversions : Both (i, j) in Y
 - Split inversions : i in X , j in Y

Algorithm: High-level idea

- CountInv (array A , length n)
 - If $n==1$ return 0
 - $X = A[1,2,\dots,n/2]$, $Y = A[n/2+1, \dots,n]$
 - $x = \text{CountInv}(X, n/2)$
 - $y = \text{CountInv}(Y, n/2)$
 - $z = \text{CountSplitInv}(X, Y, n)$
 - Return $x+y+z$



Count split inversion (conquer)



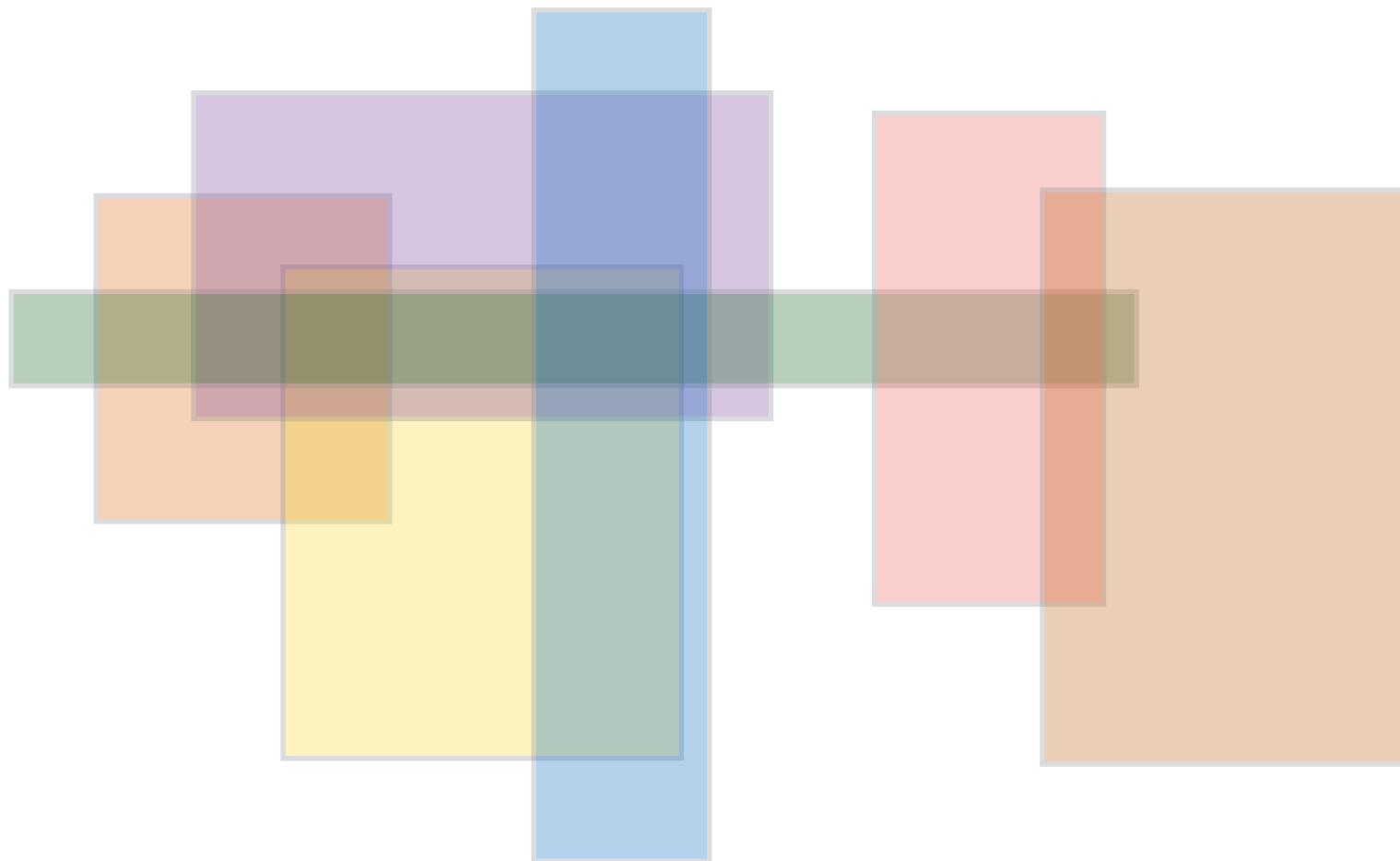
Count split inversion (conquer)

Q: How do we count split inversions in linear time?
- use Merge sort!...

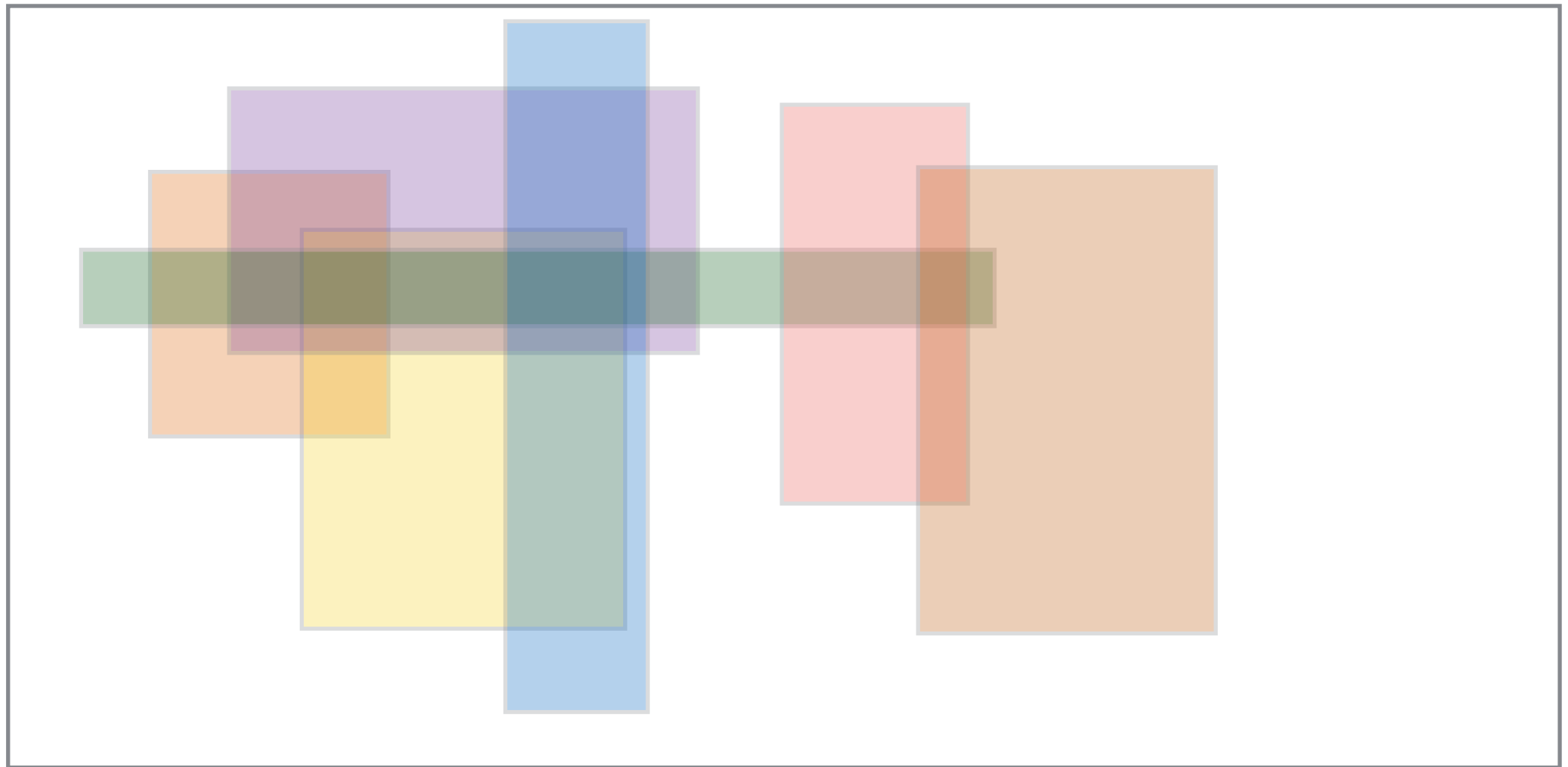
Area coverage

Input: A set of rectangles in $\mathbb{R}^2$

- Find the total area covered

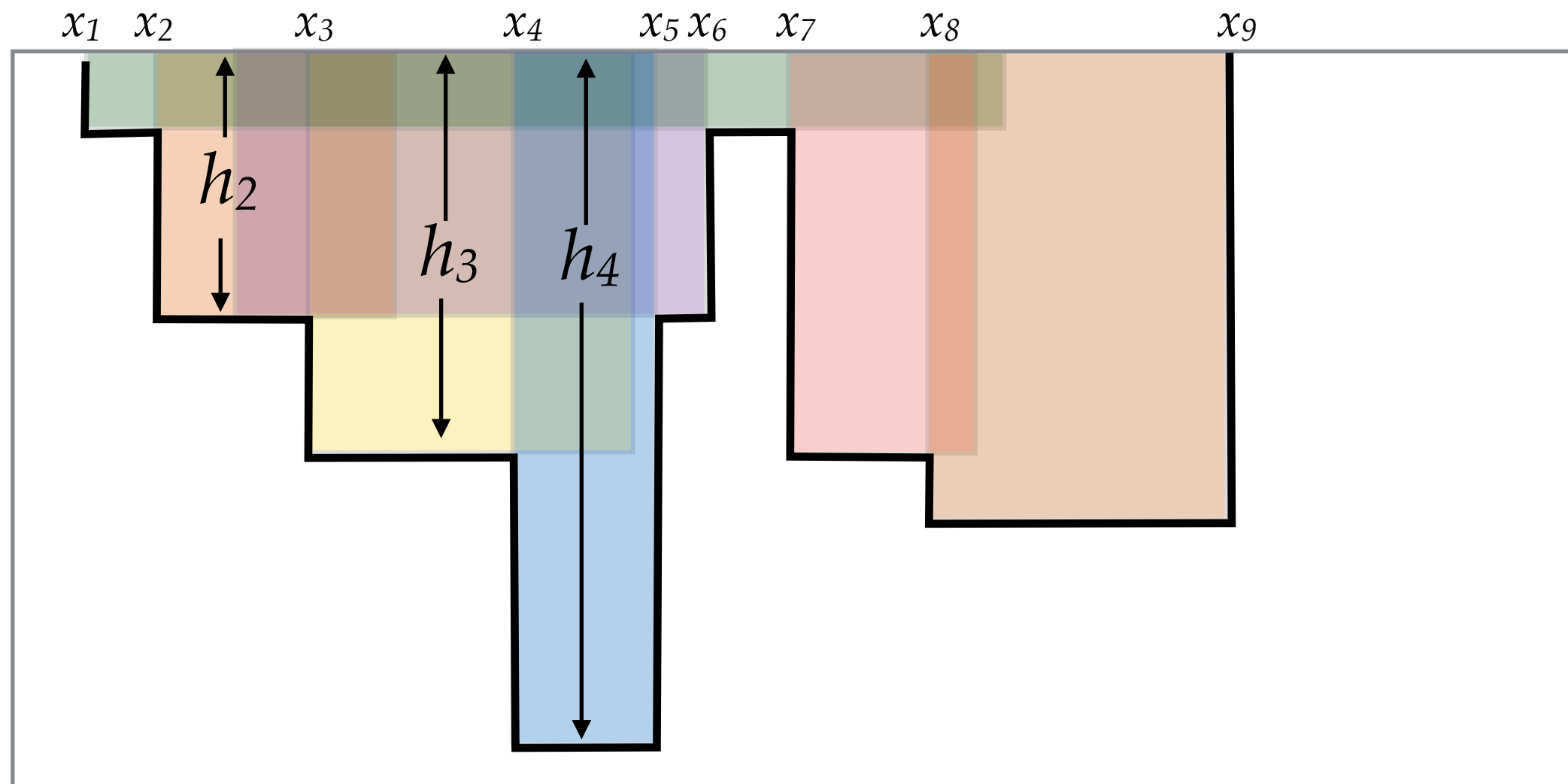


Area coverage



- Find the total area covered

Simpler version

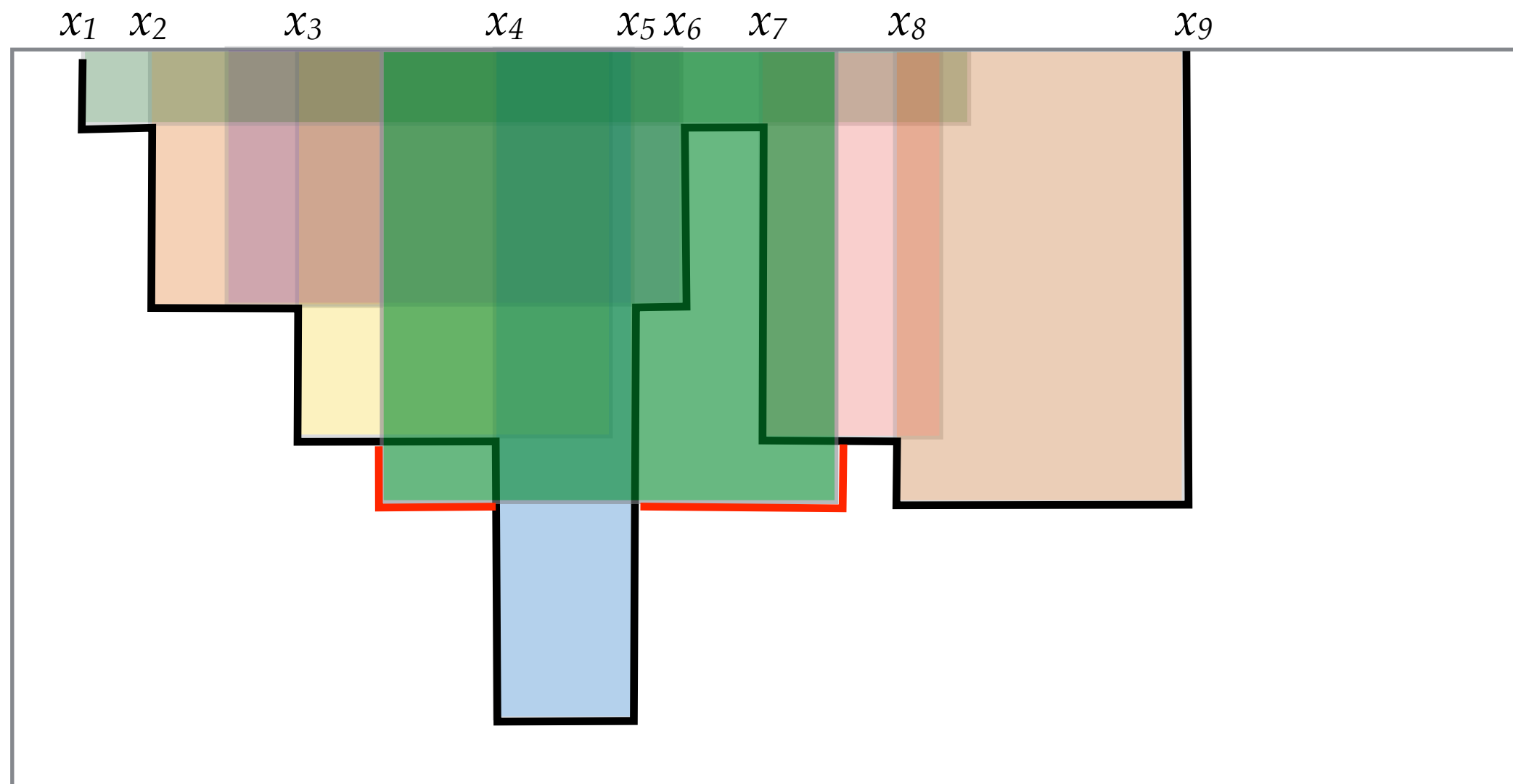


- Also known as **skyline** problem
- Input: For each rectangle (l_i, r_i, d_i) .
- Compute outline: $(x_1, h_1), (x_2, h_2), (x_3, h_3), (x_4, h_4), (x_5, h_5), (x_6, h_6), (x_7, h_7), (x_8, h_8), (x_9, 0)$,

Skyline problem: applications

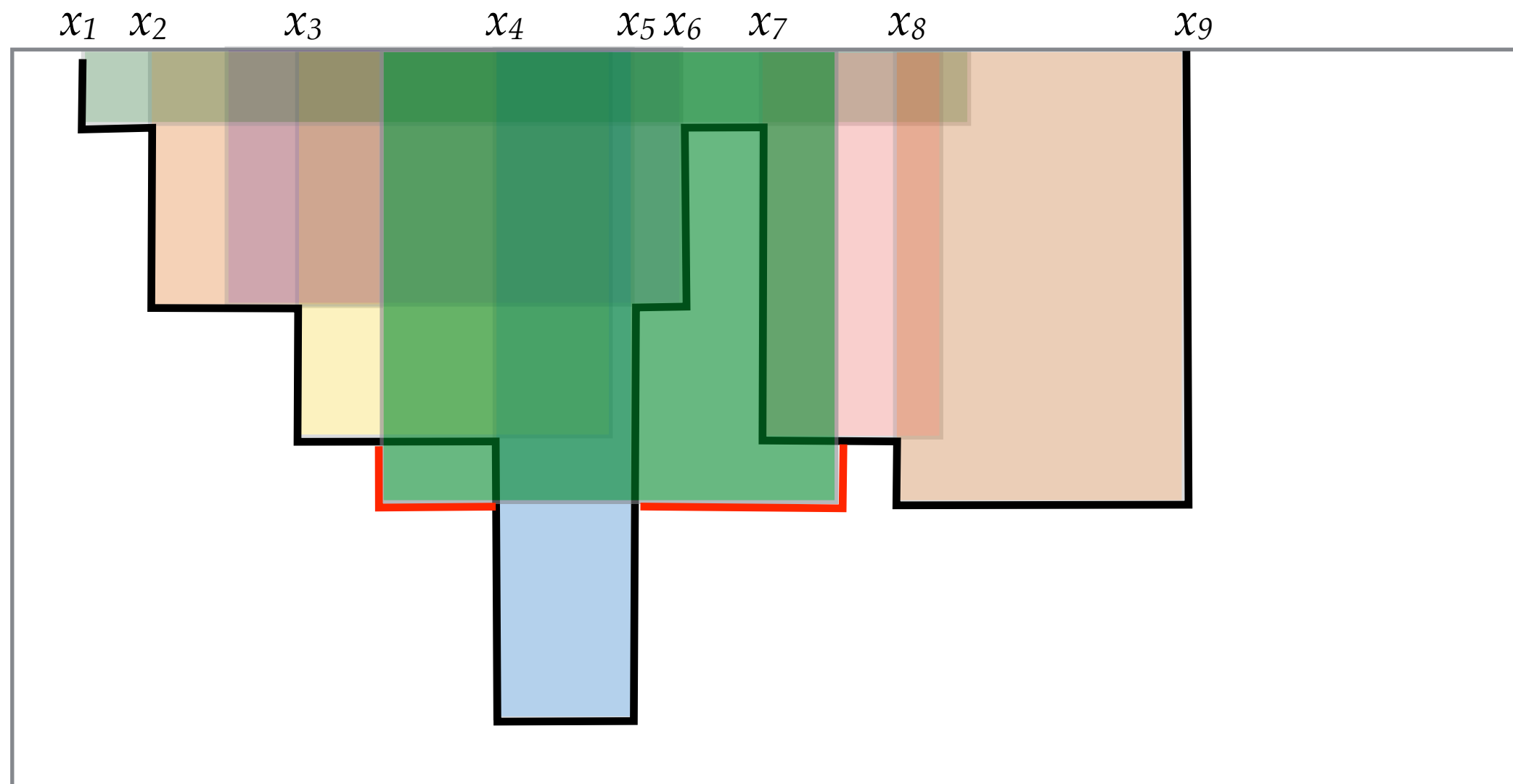
- Multi-criteria querying
 - Recommendation systems: Hotels, jobs, products...
- Preference queries in databases
 - Supports partial orders, monotonic preferences
- Interactive analysis
 - Visualisation, pruning ...
- Market segmentation, customer profiling.

$O(n^2)$ algorithm



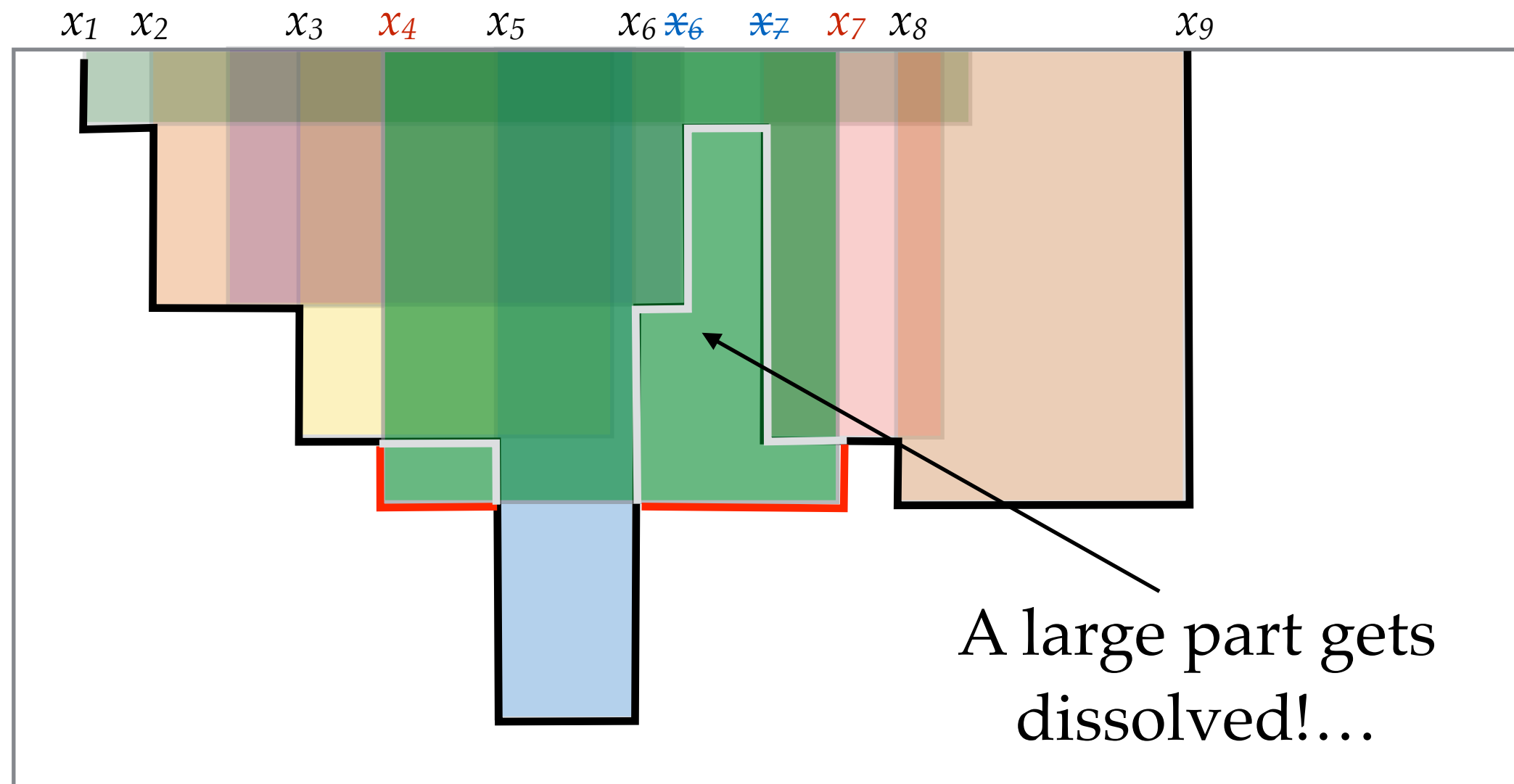
- First solution: introduce rectangles one by one, and update the outline.

$O(n^2)$ algorithm



- First solution: introduce rectangles one by one, and update the outline.
- Time: $O(n)$ per update.

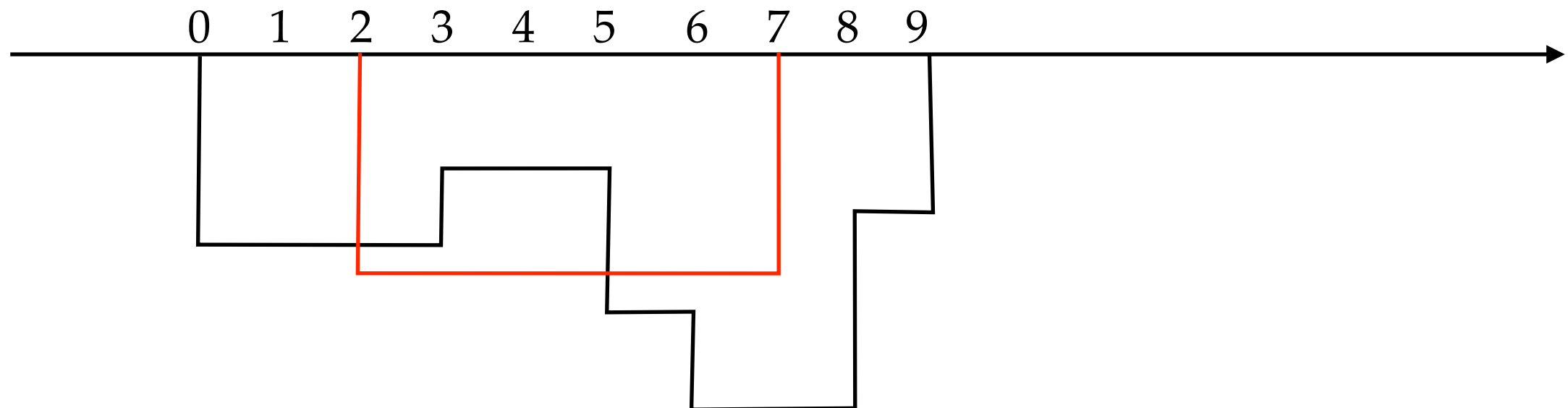
$O(n^2)$ algorithm



- First solution: introduce rectangles one by one, and update the outline.
- Time: $O(n)$ per update.

Update example

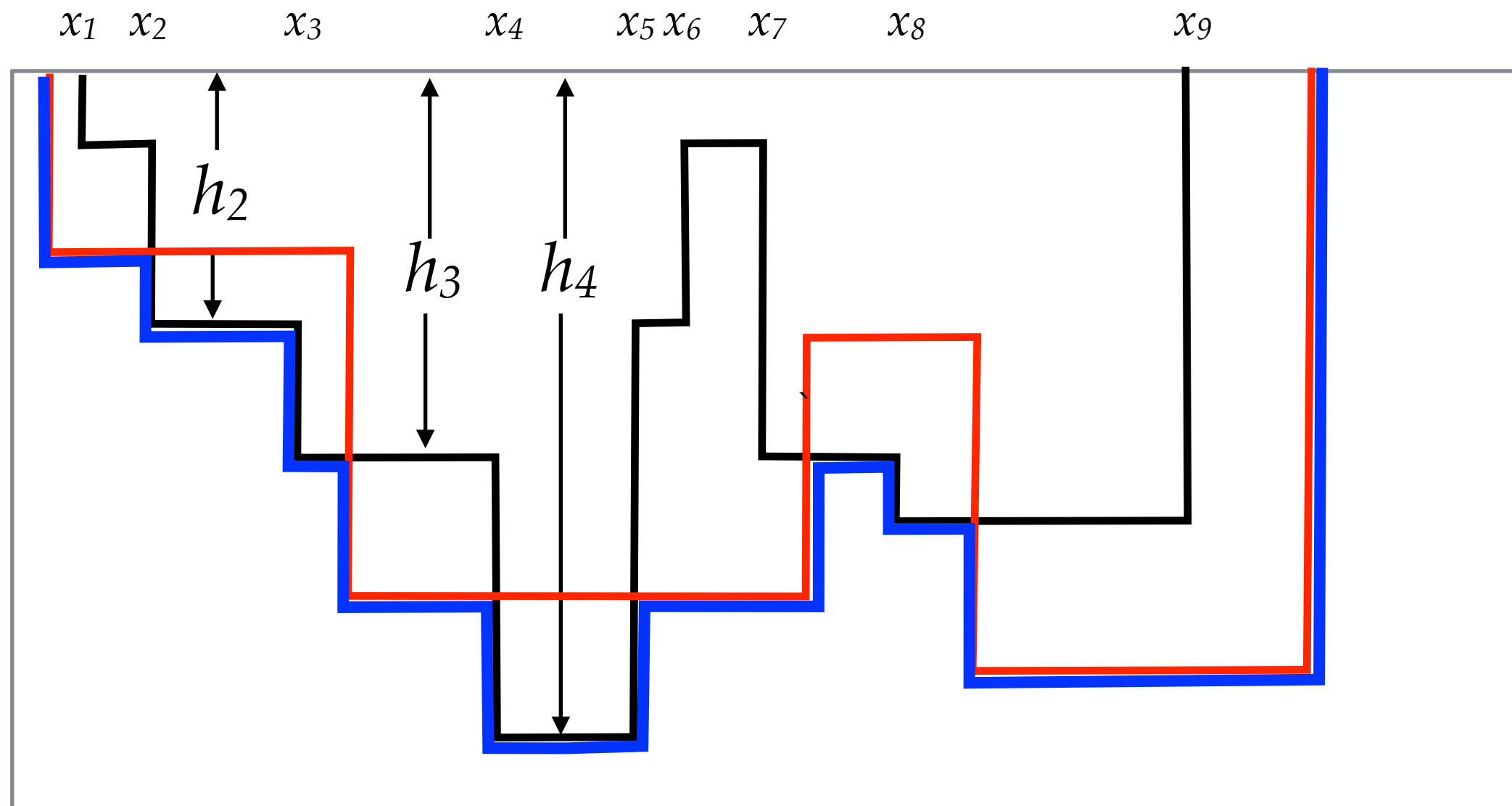
- Current outline: (0, 5), (3, 3), (5, 7), (6, 9), (8, 4), (9, 0)
- Incoming rectangle: (2, 7, 6)
- Updated outline: (0, 5), (2, 6), ~~(3, 3)~~, (5, 7), (6, 9), (8, 4), (9, 0)
- Update can be done in $O(n)$ time?



Divide and conquer approach

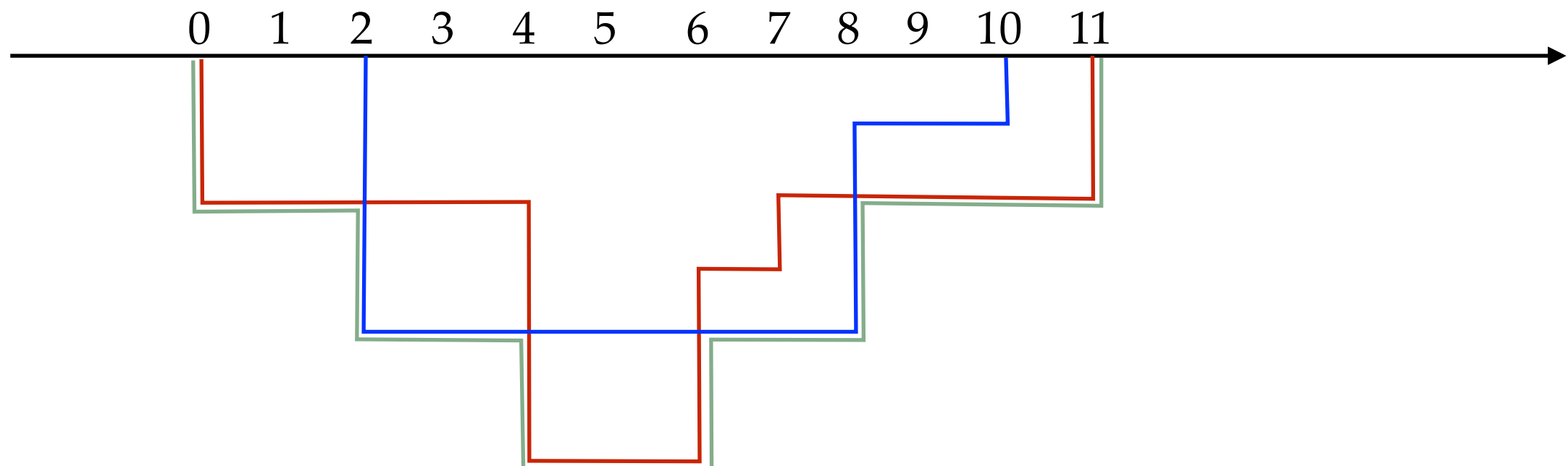
- Divide the set of rectangles into two parts with $n/2$ rectangles each.
- Suppose we have computed the outline for each set of $n/2$ rectangles.
 - Outline 1: $(x_1, h_1), (x_2, h_2), (x_3, h_3), \dots, (x_m, 0)$
 - Outline 2: $(a_1, p_1), (a_2, p_2), (a_3, p_3), \dots, (a_k, 0)$
- “merge” the two outlines to compute a new outline.

Merge two outlines



Merge two outlines

- Outline 1: (0, 2), (4, 6), (6, 3), (7, 2), (11, 0)
- Outline 2: (2, 4), (8, 1), (10, 0)
- Merged: (0, 2), (2, 4), (4, 6), (6, 4), (8, 2), (11, 0)



Merge two outlines

- Outline 1: $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, 0)$
- Outline 2: $(a_1, b_1), (a_2, b_2), (a_3, b_3), \dots, (a_k, 0)$
- Pointers for two queues i and j .
- Maintain the current height in each outline
 $height_1, height_2$

Merge two outlines

- Outline 1: $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, 0)$
- Outline 2: $(a_1, b_1), (a_2, b_2), (a_3, b_3), \dots, (a_k, 0)$
- Pointers for two queues i and j .
- Maintain the current height in each outline
 $height_1, height_2$
- If $x_i < a_j$ then
 - $height_1 \leftarrow y_i$
 - Push $(x_i, \max (height_1, height_2))$
 - $i \leftarrow i + 1$
- Else is similar

Merge two outlines

- Outline 1: $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, 0)$
- Outline 2: $(a_1, b_1), (a_2, b_2), (a_3, b_3), \dots, (a_k, 0)$
- Pointers for two queues i and j .
- Maintain the current height in each outline
 $height_1, height_2$
- If $x_i < a_j$ then
 - $height_1 \leftarrow y_i$
 - Push $(x_i, \max (height_1, height_2))$
 - $i \leftarrow i + 1$
- Else is similar
- Final clean up: remove consecutively repeating heights in the merged outline

Alternative implementation

- Maintain the current height in each outline
 $height_1, height_2$
- If $x_i < a_j$ then
 - $height_1 \leftarrow y_i$
 - $height_to_push \leftarrow \max (height_1, height_2)$
 - If $(last_pushed_height \neq height_to_push)$
 - Push $(x_i, height_to_push)$
 - $last_pushed_height \leftarrow height_to_push$
 - $i \leftarrow i + 1$
- Else is similar

Other approaches

- Divide and conquer with respect to heights?
- Approaches without divide and conquer... ?

Other approaches

- Divide and conquer with respect to heights?
- Approaches without divide and conquer
- Reordering the input
 - Increasing order of left end points: $O(n \log n)$ time implementation possible using a data structure like balanced binary tree.
 - Decreasing order of heights: $O(n \log n)$ time implementation possible using a data structure like balanced binary tree or heap.
- $O(n \log n)$ necessary ?

Significant inversion

- Given an array A of integers
- a pair (i, j) is called a significant inversion if $i < j$ and $A[i] > 2A[j]$.
- $O(n \log n)$ time algorithm to find the number of significant inversions.

Significant inversion

- Given an array A of integers
- a pair (i, j) is called a significant inversion if $i < j$ and $A[i] > 2A[j]$.
- $O(n \log n)$ time algorithm to find the number of significant inversions.
- Divide and conquer will work.

Significant inversion

- Given an array A of integers
- a pair (i, j) is called a significant inversion if $i < j$ and $A[i] > 2A[j]$.
- $O(n \log n)$ time algorithm to find the number of significant inversions.
- Divide and conquer will work.
- Alternate approach...?

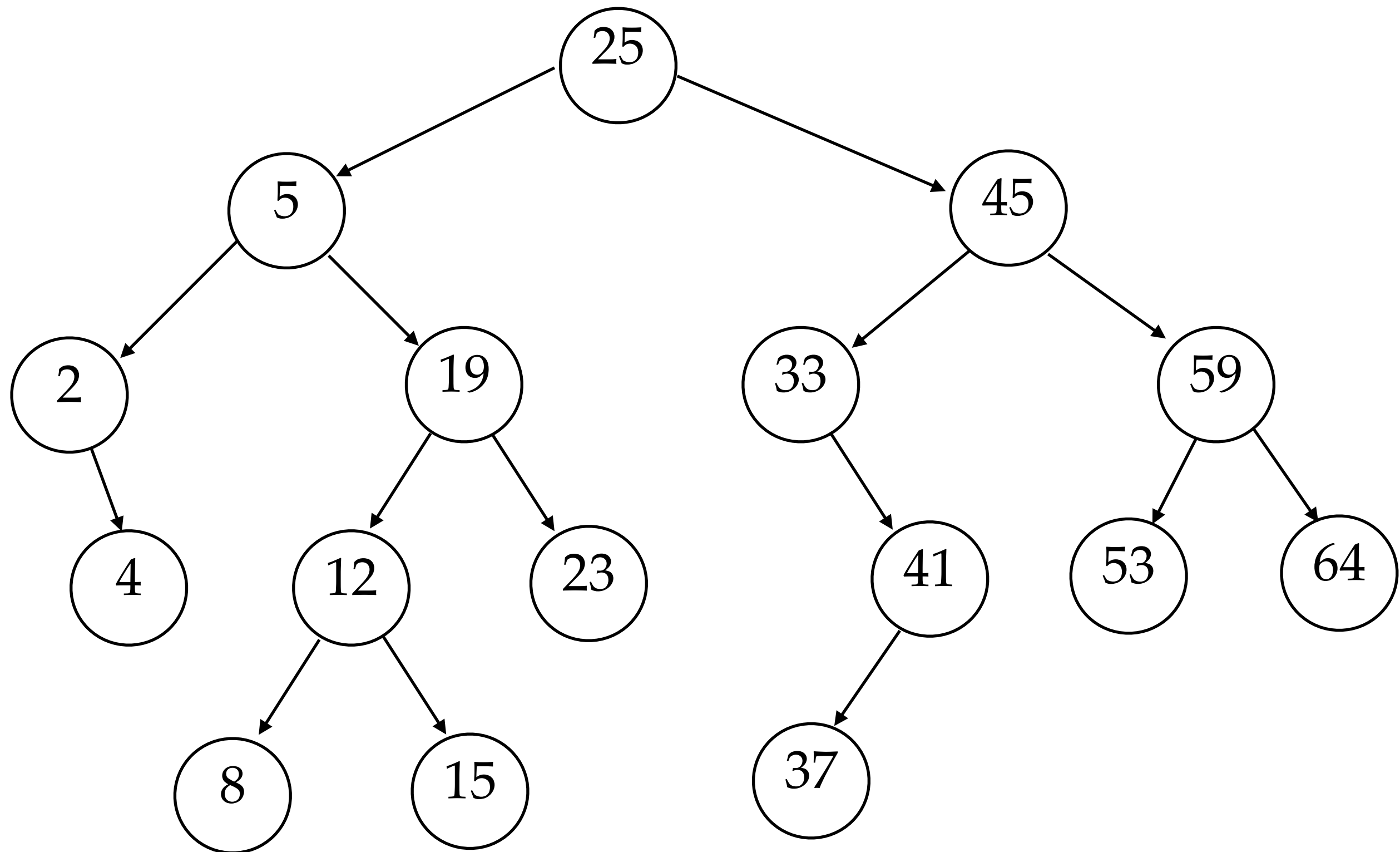
Significant inversion

- a pair (i, j) is called a significant inversion if $i < j$ and $A[i] > 2A[j]$.
- For any j , count how many are $> 2A[j]$ among first $j-1$ numbers.
- Easy to count if first $j-1$ numbers are in a sorted array
- But to maintain sorted array we will need $O(n)$ per insertion.

Significant inversion

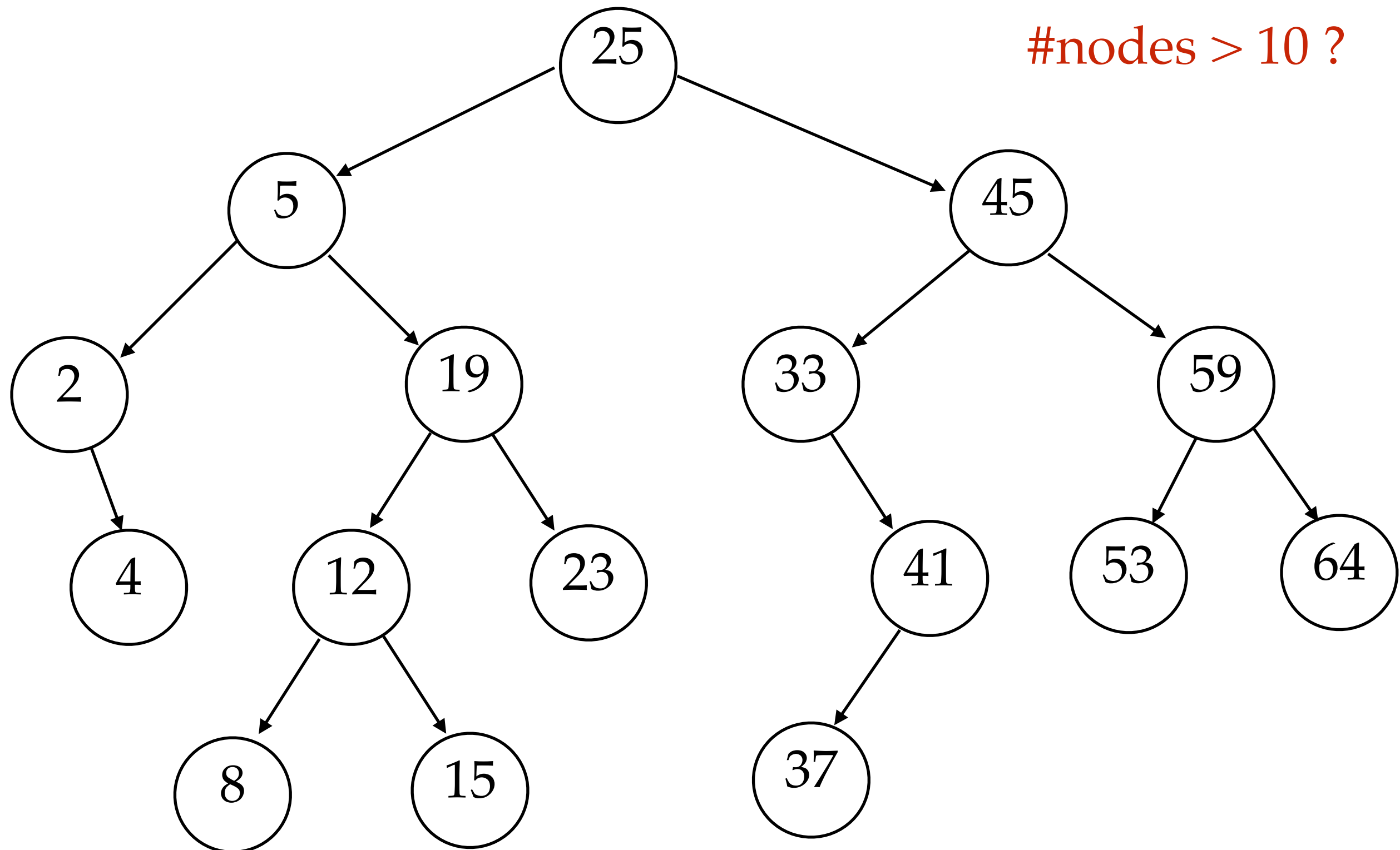
- a pair (i, j) is called a significant inversion if $i < j$ and $A[i] > 2A[j]$.
- For any j , count how many are $> 2A[j]$ among first $j-1$ numbers.
- Easy to count if first $j-1$ numbers are in a sorted array
- But to maintain sorted array we will need $O(n)$ per insertion.
- **What other data structure can be used?**

Binary search tree



Binary search tree

#nodes > 10 ?



Counting in BST

- Counting number of *nodes* $> x$ seems to take $O(n)$ time in the standard BST.

Counting in BST

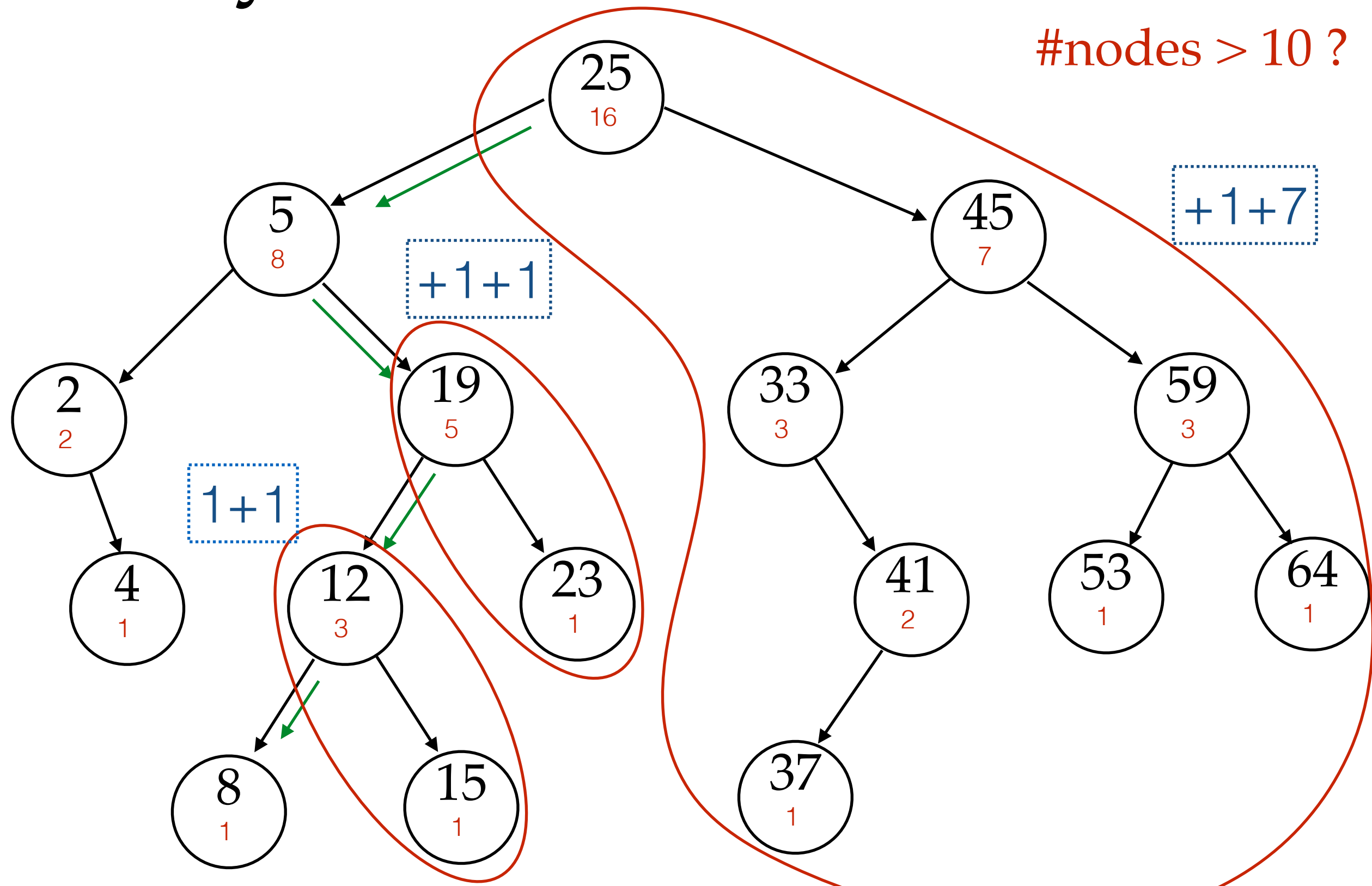
- Counting number of *nodes* $> x$ seems to take $O(n)$ time in the standard BST.
- What if store for each node, the number of nodes greater than itself ?
- Too much time to update these numbers on insertion of a new element.

Counting in BST

- Counting number of *nodes* $> x$ seems to take $O(n)$ time in the standard BST.
- What if store for each node, the number of nodes greater than itself ?
- Too much time to update these numbers on insertion of a new element.
- **Correct idea:** Store for each node x , the size of the subtree rooted at x

Binary search tree with count

#nodes > 10 ?



Counting in BST

- Store for each node x , the size of the subtree rooted at x
- Can we maintain this information on insertion of a new element?

Counting in BST

- Store for each node x , the size of the subtree rooted at x
- Can we maintain this information on insertion of a new element?
 - Need to update the counts for all nodes on the path from the root to the new node.
 - $O(\log n)$ for balanced binary tree.

Counting in BST

- Store for each node x , the size of the subtree rooted at x
- Can we maintain this information on insertion of a new element?
 - Need to update the counts for all nodes on the path from the root to the new node.
 - $O(\log n)$ for balanced binary tree.
- Can counts be maintained in the re-balance step?
- Can you do range count: $\#nodes$ in range l to r
- Other data structures?

Algorithm: Significant inversion

- Maintain a balanced BST. Initially empty.
For each node x , we need to store size of the subtree rooted at x .
- $no_of_pairs \leftarrow 0$;
- **For** $j \leftarrow 1$ to n
 - Look for the right position p for $2A[j]$ in the BST (no insertion).
 - **For** every node x on the path from p to the root:
 - **If** ($x > 2A[j]$) **then**
Add (1+ size of the subtree rooted at right child of x) to no_of_pairs
 - **Insert** $A[j]$ in the BST and increase the size count for every node on the path from root to $A[j]$ by 1

Exponentiation

- Given a and n , compute a^n .
- $a \times a \times \cdots \times a$ n times
- $n-1$ multiplications

Exponentiation

- Given a and n , compute a^n .
- $a \times a \times \cdots \times a$ n times
- $n-1$ multiplications
- $a^{16} = (((a^2)^2)^2)^2$ only 4 multiplications
- $a^{11} = (((a^2)^2)^2 \times a^2 \times a$ only 5 multiplications
- Repeated squaring technique...

Repeated Squaring

- $\text{Exp}(a, n)$:
 - If n is even
 - return $(\text{Exp}(a, n/2))^2$
 - If n is odd
 - return $(\text{Exp}(a, (n-1)/2))^2 \times a$
 - If n is 1
 - return a

Number of multiplications

$$T(n) \leq T(n/2) + 2$$

$$T(n) \leq 2 \log n$$

Another implementation

- $\text{Exp}(a, n)$:
 - If n is even
 - return ($\text{Exp}(a^2, n/2)$)
 - If n is odd
 - return ($\text{Exp}(a^2, (n-1)/2)$) $\times a$
 - If n is 1
 - return a

Iterative implementation

- Input: a, n
- Initialize
 - $a_power_n \leftarrow 1;$ // this will be a^n at the end
 - $a_two_power \leftarrow a;$ // this will be a^{2^i} after i iterations
- while ($n > 0$)
 - If (n is odd) then $a_power_n \leftarrow a_two_power \times a_power_n;$
 - $a_two_power \leftarrow a_two_power \times a_two_power;$
 - $n \leftarrow n/2$ //integer part after division by 2 //or right-shift by 1 bit

Repeated squaring

- Does it give the minimum number of multiplications?
- What about a^{15} ?
 - can be done in 5 multiplications
- Given n , what the minimum number of multiplications required for a^n ? No easy answer.
- Can apply repeated squaring for other operations like Matrix powering.
- **Apparently proposed by Pingala (200 BC ?).**

Fibonacci numbers

- $F(n) = F(n-1) + F(n-2)$
- Used by Pingala to count the number of patterns of short and long vowels.
- Here again we are repeating the same operation n times.
- Can repeated squaring be used?